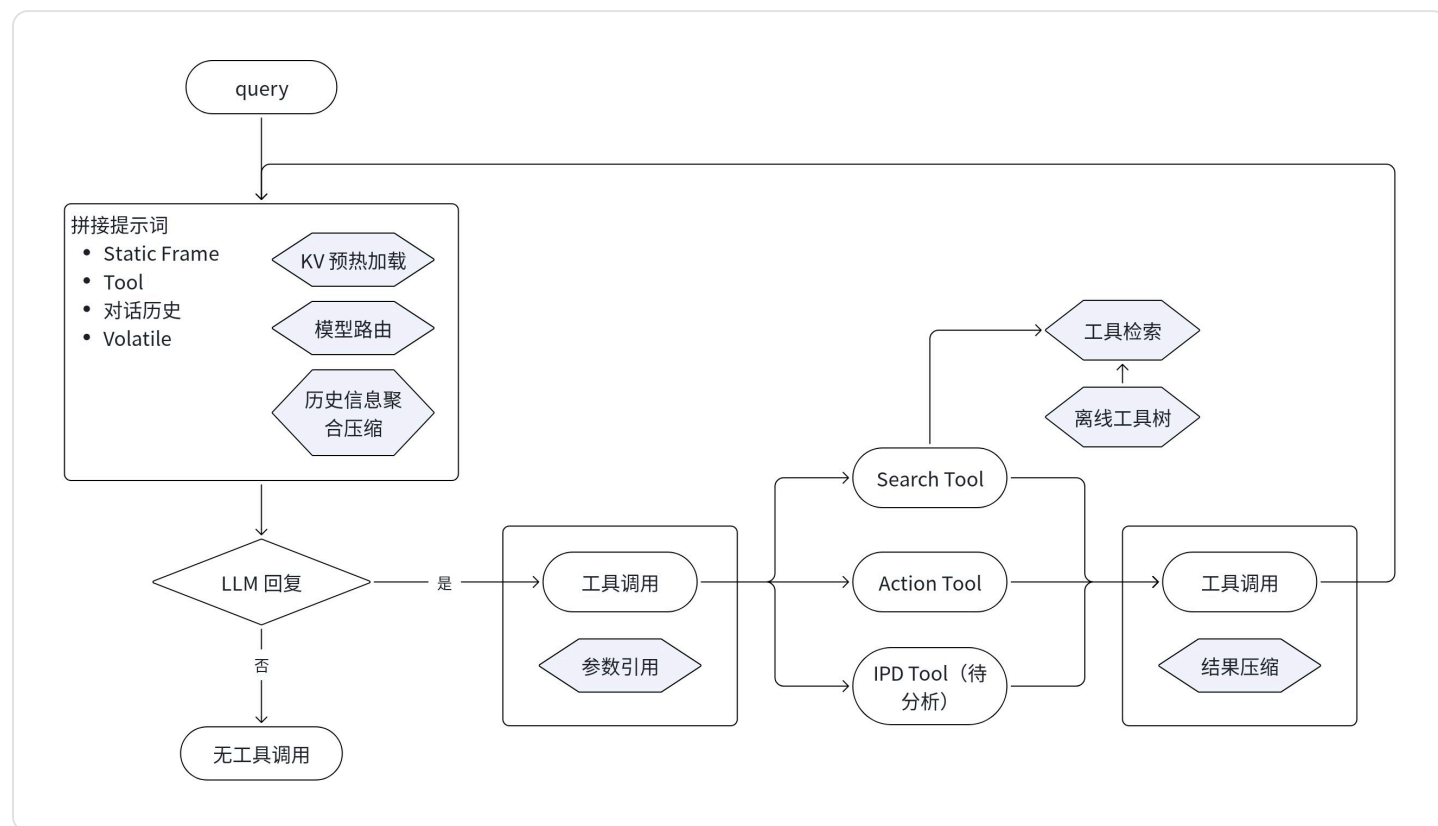


极简 Agent 设计方案

极简的逻辑在于模块划分明确，不额外增加功能。

Agent 核心通过拼接 prompts 方式与模型配合，最大限度地发挥模型能力。



Agent 内容分类

- Static Frame (写到 system prompts)
 - Agent 是谁、干嘛的
 - safety: 一句 “危险/敏感操作先确认”
 - 约束信息
 - 工具使用规则/检索引导
 - 记忆使用规则/检索引导
 - 当前工作目录路径
 - 前文索引--可对应记忆--截断历史的时候使用
- Tool: 外部操作功能
 - MCP Server 不暴露, 只对于 LLM 暴露 Tool
 - 加载方式: 1) 预置 Tool; 2) ToolSearch

- 对话历史: 保留 K 轮
 - 本质描述的是 Agent 所处环境
- Volatile 尾部 (可选, 每轮重生)
 - Runtime 时间
 - Git status (可选, 如果在 git 仓库中)

工具

工具分为三类: 1) 信息查询类 (Search Tool); 2) 外部操作类 (Action Tool); 2) 流程规划类 (IPD Tool)

信息查询

查询信息分两级:一类是查找文件,二类是基于查找到的文件 grep 相关信息

- 长期记忆重要信息检索: Memory 处理
- Data 检索 (sql 类): 查表 + 查数; 分为针对 Excel / 数据库 / DataAgent(结构化); 同时有 data 处理类 SKILL
- 外部网址检索与抓取: SearchAgent(非结构化)
- 本地文件检索: 本地文件查找 (是否需要对于本地文件做摘要); 本地文件关键信息检索(文件查找可以通用,但是打开功能需要增加); 用户输入当做本地文件
普通 vim 能打开的 / PDF / Excel / Word / PPT / HTML / 代码类
- 流程检索(SKILL检索): SKILL查询, 渐进式披露
 - SKILL渐进式披露不是写到系统提示词里, 而是写到 skill_search 工具的 result 里面
 - 渐进式披露主要是有个小型的 SearchAgent, 依据任务异步加载新的步骤, 而不是一次性都加载完毕
 - 主 SKILL最好保留目录框架
- 工具检索: 基础模块, MCP 的工具也通过工具检索按需加载

设计理念: SKILL 与 Tool 的未来趋势, SKILL 将来会是开源生态, Tool(包括 MCP) 会是各家的闭源生态, 作为技术竞争力大意外提供 Tool API/SDK, 进行收费管理. SKILL 会为 Tool 引流. Tool的参数描述如何调用, 这个需要写到 Tool 内部, 但是 Tool 的 description 会外化到外部 SKILL. 如果工具之间有确切的逻辑调用关系, 则工具之上的外包 shell 脚本使用, 不会增加 workflow 的使用方式.

外部操作

- 调用 API: 外部 API, 如查天气, 查时间等
- 写文件
- 回滚文件
- 生成类代码: shell, bash, SubAgent (参数为语言)

流程规划

IPD workflow: IPD 的流程模式(待构造 [📄 基于组织流程控制的多智能体长程任务范式](#))

工具调用要补齐能力

- 参数输入使用代码方式输入, LLM 生成时部分参数使用变量引用方式
- 工具检索结果超长的情况下使用 CompactAgent 进行压缩, 原始数据信息入库备二次查询

对话历史

对话历史主要保留 K 轮, 可以支持基于聚合方式压缩对话历史